

Generative AI Analysis Report

Conditional Transformer-Based Generation of 2D Platformer Level Segments

Student: Robert Mayfield **Project:** Udacity AI Masters Capstone, Generative AI Applications

Overview

This project implements and evaluates a conditional generative model for 2D platformer level segment synthesis. The work sits within the broader field of procedural content generation, the algorithmic creation of game content such as levels, maps, and items (Shaker et al., 2016). The model is a decoder only Transformer trained on tile based level data from the Video Game Level Corpus (VGLC), with the objective of generating novel level segments that align with a requested difficulty target: Easy, Medium, or Hard. Difficulty conditioning is implemented through prepended control tokens that prompt the model to generate structurally appropriate tile sequences.

The primary research question is whether a small autoregressive Transformer can learn the structural associations between difficulty labels and tile configurations well enough to produce directed generation, given the small corpus size available in the VGLC. Secondary questions concern the effect of corpus size on generation quality and the effectiveness of alternative sampling strategies for improving output novelty.

The project trains a conditional model and a baseline unconditional model, evaluates them on four metrics (structural validity, diversity, nearest neighbour similarity, and difficulty match accuracy), and extends the analysis with a corpus expansion experiment and a sampling strategy experiment. Key results show strong difficulty alignment (94.0% baseline accuracy, 95.3% with the expanded corpus) but near verbatim reproduction of training data in most outputs, with mean nearest neighbour tile similarity around 0.99.

Dataset Description

The primary data source is the Video Game Level Corpus (VGLC), an open academic dataset of tile based platformer levels (Summerville et al., 2016). The VGLC encodes each level as a grid of tile characters representing gameplay elements: ground, enemies, pipes, blocks, gaps, and collectibles. This project uses Super Mario Bros (SMB1) overworld levels as the baseline training corpus, with Super Mario Bros: The Lost Levels (SMB2J) added in the corpus expansion experiment.

Baseline corpus: 15 SMB1 overworld levels downloaded from the VGLC repository. Levels range from 149 to 373 columns wide and are uniformly 14 rows tall. Total tile count across the corpus is 40,922 tiles.

Expanded corpus: 22 Lost Levels files were downloaded from the VGLC. Of these, 20 were retained after normalization: files with non standard row heights had leading all sky rows trimmed to reach the 14 row standard, and 2 files (12 and 13 rows) that could not be normalized by sky trimming alone were excluded. The expanded corpus contains 35 levels total.

The tile vocabulary contains 13 characters: the empty sky tile (-), ground (X), breakable blocks (S), enemies (E), pipes (<, >, [,]), question blocks (?, Q), cannon tiles (B, b), and coins (o). The sky

tile dominates at 86.21% of all tile positions. Ground tiles account for 8.80%, and all interactive and hazard tiles together account for less than 3%.

Each level is split into overlapping 14 row by 32 column chunks with a stride of 16 columns. This produces 161 chunks from the baseline corpus and 390 chunks from the expanded corpus. Each chunk contains 448 tile positions and represents approximately two screens of horizontal game-play.

The dataset is committed directly to the repository under `data/raw/` and requires no external download to reproduce the analysis.

Model Design and Training Approach

Architecture. Both the conditional and baseline models use a decoder only Transformer architecture for next token prediction (Vaswani et al., 2017). The model consists of 4 Transformer encoder layers with 8 attention heads, a model dimension of 256, and a feedforward sublayer of 1024 dimensions. The output projection maps from 256 dimensions to the 16 token vocabulary (13 tile characters plus 3 difficulty control tokens). Total parameter count is approximately 3.28 million for both models. In PyTorch this is implemented using `nn.TransformerEncoder` with a causal self attention mask (`is_causal=True`), which produces behavior functionally identical to the masked self attention decoder of the original Transformer (Vaswani et al., 2017), without requiring a separate encoder input.

This architecture follows the approach of Sudhakaran et al. (2023), who demonstrated that autoregressive Transformers can generate playable Mario style level segments by treating level content as a flat token sequence. The decoder only formulation is appropriate because level generation is an autoregressive task: each tile is predicted from all preceding tiles in a left to right, top to bottom reading order.

Tokenization. Tile characters are assigned integer IDs 0 through 12 in sorted order. Three difficulty control tokens are assigned IDs 13 through 15 (Easy, Medium, Hard). Each tokenized training sequence has length 449: one control token followed by 448 tile tokens read row by row across the 14 row by 32 column chunk.

Conditioning mechanism. The conditional model receives a difficulty control token as its first input token for every training sequence. The token is prepended before the tile content and is therefore attended to by every subsequent tile prediction through the self attention mechanism. At generation time, the model is seeded with a control token and generates tile tokens autoregressively until the full 448 tile sequence is complete. Control tokens are masked from the tile prediction distribution at every step to prevent them from appearing as tile outputs.

The baseline model is trained on the same tile sequences with the control token stripped, providing a reference point for quantifying the effect of conditioning.

Training. Both models are trained for 150 epochs using the AdamW optimizer (Loshchilov & Hutter, 2019) with a learning rate of $3e-4$, weight decay of $1e-2$, and gradient clipping at a maximum norm of 1.0. The loss function is cross entropy over the 16 token vocabulary. The train/validation split uses an 80/20 stratified split preserving difficulty class balance. For the baseline corpus this produces 128 training and 33 validation sequences; for the expanded corpus, 312 training and 78

validation sequences. All random seeds are fixed at SEED=42 using Python, NumPy, PyTorch CPU, PyTorch CUDA, and cuDNN determinism flags.

Preprocessing Decisions

Difficulty scoring. Difficulty labels are derived from five structural features computed per chunk: enemy density (fraction of tiles that are enemies), hazard density (fraction of tiles that are enemies or cannon tops), longest ground gap (maximum consecutive empty tile run in the bottom row), solid tile ratio (fraction of tiles that are solid), and collectible density (fraction of tiles that are coins). A weighted difficulty score is computed as:

$$\text{score} = \text{enemy_density} \times 40 + \text{longest_gap} \times 5 + \text{hazard_density} \times 30$$

The longest ground gap carries the dominant weight because pit navigation is the primary mechanical challenge in Super Mario Bros gameplay. The choice of structural features such as enemy density, gaps, and hazards as difficulty signals follows established practice in the procedural content generation via machine learning literature (Summerville et al., 2018). The feature weights are heuristic and exploratory rather than empirically validated. This reflects a broader reality in the field: there is no consensus methodology for evaluating procedural level generation systems, and difficulty in particular lacks a standard quantitative definition (Withington et al., 2024). The scoring formula here is therefore presented as one reasonable heuristic among many rather than a ground truth measure of difficulty.

Difficulty labels are assigned using percentile based thresholds: chunks scoring below the 33rd percentile are labeled Easy, chunks between the 33rd and 67th percentile are labeled Medium, and chunks at or above the 67th percentile are labeled Hard. For the baseline corpus the thresholds are Easy < 5.68 and Hard >= 20.78, producing 54 Easy, 52 Medium, and 55 Hard chunks.

Percentile based thresholds were chosen over fixed thresholds to ensure balanced class sizes regardless of corpus composition. With only 161 baseline chunks and 3 classes, severe class imbalance would prevent the conditional model from learning meaningful difficulty associations.

Corpus normalization. Lost Levels files use variable row heights (12 to 16 rows). All sky leading rows were trimmed from 15 row and 16 row files to reach the 14 row standard. This normalization is semantically clean because all removed rows consisted entirely of the sky tile with no content. Files with 12 or 13 rows were excluded because they required padding rather than trimming, which would introduce synthetic tile content.

Output Evaluation and Interpretation

Structural validity was 100% across all experiments. Every generated segment used only the 13 valid tile characters, produced the correct 14 row by 32 column grid, and contained at least one non empty tile. This result is expected: the generation loop masks control token IDs from the tile prediction distribution at every step, making out of vocabulary generation impossible by construction.

Difficulty match accuracy was 94.0% overall for the conditional model on the 15 level corpus, with per class F1 scores of 0.933 (Easy), 0.915 (Medium), and 0.970 (Hard), and a macro F1 of

0.940. These results are well above the 33% chance baseline and confirm that the difficulty control token has a measurable effect on generated output structure. Medium achieved the lowest F1 because it occupies the interval between both score thresholds. Its precision was high (0.98) but its recall lower (0.86), meaning Medium conditioned outputs often drift into an adjacent class even though anything scored Medium is almost always correct.

Expanding the corpus to 35 levels raised overall accuracy to 95.3% and macro F1 to 0.953, with per class F1 rising for Easy (0.962) and Medium (0.926) and holding for Hard (0.970). One caveat is essential to this comparison: each model is scored against difficulty thresholds computed from its own corpus (Easy < 5.68 and Hard >= 20.78 for the baseline, Easy < 10.74 and Hard >= 40.94 for the expanded corpus), so the two figures are not measured on a common scale. The 1.3 percentage point gain therefore reflects both the additional structural variety the Lost Levels provide and the recalibrated thresholds, not corpus size alone.

Diversity was 78.0% for the conditional model on the 15 level corpus: 117 of 150 generated segments were distinct tile grids. Expanding the corpus improved diversity slightly to 79.3%.

Nearest-neighbour similarity revealed the extent of memorization. The mean maximum tile similarity between a generated segment and its closest training chunk was 0.990 for the conditional model on the 15 level corpus, meaning generated outputs differed from their nearest training match in approximately 4 to 5 tile positions out of 448. Expanding the corpus raised mean similarity slightly to 0.993 because the larger corpus covers more structural space, making it more likely that any generated output finds a close training match.

Sampling experiments showed that temperature is the most effective lever for reducing memorization behavior. At temperature 1.2, diversity reached 89.3% and mean max similarity dropped to 0.986 while difficulty match accuracy held at 97.3%, matching the temperature 1.0 setting. At temperature 1.5, diversity reached 98.0% and mean max similarity dropped to 0.957, but accuracy fell to 88.0%. The novelty rate, the fraction of outputs more than 10% different from any training chunk, rose from 0% at temperature 0.8 to 14.7% at temperature 1.5, the only configuration that produced a meaningful share of genuinely new structure. Temperature 1.2 represents the best practical trade off for applications requiring varied output without severe accuracy degradation.

Top-k sampling produced modest improvements over the temperature 1.0 baseline with minimal accuracy impact, though no configuration meaningfully reduced mean max similarity. Nucleus sampling (top-p; Holtzman et al., 2020) was counterproductive for this vocabulary: at low p values the sky tile dominates the probability nucleus, removing structural tiles from the sampling distribution and collapsing diversity to 6% at p equal to 0.80. This is a direct consequence of the 86% sky tile frequency and is specific to heavily skewed tile distributions.

A model capacity experiment tested the central claim that memorization is driven by capacity exceeding the available data. Two smaller models were trained on the same 15 level corpus and identical hyperparameters and evaluated on the same metrics. The result is a clean monotonic curve. As capacity grows from 81 thousand to 458 thousand to 3.28 million parameters, difficulty match rises (0.380, 0.540, 0.940) and mean similarity rises with it (0.908, 0.930, 0.990), while diversity and novelty fall. Reducing capacity does reduce memorization, which confirms the mechanism, but the smallest model still reproduces about 91% of its nearest training chunk and only got there by collapsing difficulty match to 0.380, near the 0.333 chance floor for three balanced classes. There is no smaller model that both conditions well and avoids memorizing. Together with the corpus expansion result, where more data did not escape memorization either, this points past model size to data scale, and specifically the availability of structurally diverse data beyond

the Mario game family, as the binding constraint under these training conditions.

Interpretation for a Non-Technical Audience

This project taught a computer program to create new platformer level sections by learning from existing Super Mario Bros levels. The program studied 35 levels from two Mario games and learned patterns about how tiles are arranged: where the ground is, where enemies appear, where gaps occur. It can then generate new level sections by predicting, one tile at a time, what should come next based on everything it has already placed.

The program can be given a difficulty instruction before it starts generating. When told to make an Easy level, it tends to produce sections with solid, unbroken ground and few obstacles. When told Hard, it tends to produce sections with longer ground gaps that require bigger jumps. This difficulty control works about 94% of the time on the original training data and about 95% of the time after adding the Lost Levels, when measured against the same scoring system used to label the training data.

The most important limitation is that the program mostly reproduces patterns it has already seen rather than inventing truly new designs. When we compared its outputs to the training levels tile by tile, we found that nearly every output was more than 99% identical to some chunk from the training data. The program is working more like a very organized filing system than a creative designer: it retrieves appropriate memorized patterns rather than composing new ones. This happens because the training dataset is small and the program has enough capacity to remember most of it.

Increasing the randomness in how the program makes tile selections (higher temperature) does produce more genuinely different outputs, though it also makes the difficulty control less reliable. Adding more training data from the Lost Levels improved the variety of patterns available, but did not change the fundamental tendency toward reproduction.

Failure Cases and Error Analysis

Four categories of failure were identified through systematic analysis of 150 generated segments.

Difficulty misclassification. The failure probe generated 90 samples (30 per class) and surfaced five misclassifications (5.6%), consistent with the difficulty match accuracy reported in evaluation. Three were Medium conditioned. Two of those scored Easy, with scores of 0.31 and 0.85, because their ground rows were essentially solid and their few enemies and blocks did not lift the score past the Easy threshold of 5.68, and one scored Hard at 30.07, above the Hard threshold of 20.78, because its ground row carried enough of a gap. The other two were Hard conditioned and both fell short of the Hard threshold of 20.78, one scoring Easy at 5.47 and one scoring Medium at 20.16. No Easy conditioned output was misclassified. This is a threshold effect: an output whose computed score lands near a percentile threshold is assigned to an adjacent class regardless of the conditioning token. The underlying cause is structural: the model has not learned a semantic notion of difficulty, only structural associations with control tokens that are mediated entirely by the scoring formula, so any output whose score crosses a threshold is relabelled.

Near-verbatim reproduction. Two reproduction examples were found, both exact copies. An Easy conditioned output matched a chunk from mario-6-2 column 0 at 1.0 similarity with zero differing tiles, and a Medium conditioned output matched a chunk from mario-4-1 column 144, also a perfect tile for tile copy. This is consistent with the 0.990 mean nearest neighbour similarity measured in the evaluation section. The model has memorized specific training sequences and retrieves them under the appropriate conditioning context rather than composing new arrangements.

Pipe integrity violations. Seven of 150 samples (4.7%) contained pipe top tokens (<>) without pipe body tokens ([]) directly below, producing structures that are tile vocabulary valid but semantically impossible to render correctly. These violations arise from the autoregressive sampling process: the model predicts a pipe top token at a row where the training distribution supports it, but subsequent row predictions choose other content rather than a pipe shaft. Because the structural validity check only verifies tile vocabulary and grid dimensions, these violations pass validation.

Scoring weight artifact in Easy samples. Twelve of 50 Easy scored outputs (24%) contained six or more enemy tiles. The worst example had 12 enemies with a solid ground row, scoring Easy because the longest gap was zero. This is a direct consequence of the heuristic scoring design: the gap feature dominates the difficulty score by a wide margin over enemy density at typical corpus values. The model has learned that Easy conditioning correlates with solid ground rows and with certain structural patterns from the training data, some of which include high enemy counts.

Ethical Considerations and Responsible Use

Data provenance and copyright. The VGLC levels are derived from commercial video games owned by Nintendo. The VGLC is published for academic research purposes under a Creative Commons license, and its use here is consistent with that context. The levels are not reproduced or redistributed in a form that would substitute for the original games. Generated outputs are new tile grids derived from learned structural patterns, not copied level content, though the memorization analysis showed that a substantial fraction of outputs are near identical to training chunks.

Bias in training data. The corpus is drawn entirely from two Nintendo titles from the mid-1980s, both designed for the same hardware platform and following the design conventions of a single creative team. Generated outputs reflect these conventions and cannot be expected to represent the full space of platformer level design. Levels designed for accessibility, for different demographic audiences, or for contemporary design conventions are absent from the training distribution. Any system that uses this generator to make decisions about player experience should account for the narrow design vocabulary it has learned.

Heuristic difficulty labels. The difficulty labels used to train the conditional model are computed from structural features, not from player data. A level labeled Hard by this system may not feel challenging to an experienced player, and a level labeled Easy may be impassable for a player with limited motor skills or game experience. The labels encode a specific technical definition of difficulty tied to gap length and enemy density, not a general psychological or accessibility based assessment. This distinction must be communicated clearly in any downstream application that surfaces difficulty labels to users or uses them to make adaptive decisions.

Deployment considerations. The model is intended as a research prototype for exploring conditional generation. It should not be deployed in a production system without human review of generated outputs, validation against a broader set of difficulty criteria, and disclosure to end users

that content is algorithmically generated. Automated suppression or promotion of generated level content based solely on classifier output without human oversight would be inappropriate. The memorization behavior identified in evaluation means that deployed outputs may closely replicate training data, raising additional questions about appropriate use in commercial contexts. Documenting the model’s intended use, training data, and known limitations in the manner of a model card would support responsible deployment (Mitchell et al., 2019).

Limitations

Corpus size and memorization. The most significant limitation is memorization, and the capacity experiment shows it is governed by the ratio of model capacity to data rather than by model size alone. With 128 to 312 training sequences and a 3.28 million parameter model, the model has sufficient capacity to memorize training data rather than learn generalizable structural rules. Mean nearest neighbour tile similarity of 0.99 at the default sampling temperature confirms this, and only aggressive high temperature sampling reduces it appreciably. The experiment also shows the limit is not fixable by tuning the model: shrinking it lowers memorization only by sacrificing difficulty conditioning, and the corpus expansion showed that adding the available VGLC data does not escape it either. Both levers fail within the resources at hand, which points to data scale as the binding constraint. A masked loss approach, removing the sky tile from the training signal while retaining it as context, is a candidate near term experiment that requires no architectural changes and could redirect model capacity toward the 14% of tiles that define level structure.

Heuristic difficulty scoring. The gap dominated scoring formula creates a systematic mismatch between the difficulty label and intuitive difficulty: Easy labeled outputs can contain many enemies, and Hard labeled outputs without gaps can be structurally benign. Player validated difficulty labels from playtesting data would be a more reliable signal, but no such dataset exists for the VGLC corpus at the chunk level.

Vocabulary skew. The 86% sky tile frequency creates practical problems for sampling: nucleus sampling is rendered ineffective because the sky tile dominates the probability nucleus at any reasonable p value. This limits the applicability of standard language model decoding strategies without modification. A run length encoding of sky regions or a masked loss training approach could reduce this imbalance.

Single-game domain. All training data comes from two games sharing the same engine. The model has learned Mario specific structural conventions (horizontal ground rows, specific pipe and pipe body paired tiles, specific enemy placement conventions) that may not transfer to other platformer styles. Expanding to other VGLC games or to custom designed levels would broaden the model’s structural vocabulary, though it would require handling vocabulary differences across games.

Generation coherence. The autoregressive model predicts tiles one at a time without explicit awareness of long range structural constraints. Pipe tops should always be followed by pipe bodies; ground gaps should be jumpable given the player’s physics parameters; enemy placements should relate to platform positions. The model captures some of these regularities through statistical co-occurrence but can produce locally valid but globally inconsistent structures, as seen in the pipe integrity violations identified in the failure analysis.

Future Improvements

Simpler data efficient architectures. The memorization result invites the question of whether a lower capacity method would fit this small corpus better. Methods that learn only local statistics and recombine them, such as n-gram slice models, multi dimensional Markov chains, and constraint based generators like WaveFunctionCollapse, carry a tiny parameter footprint with almost nothing to overfit, and they were developed for exactly this small corpus Mario setting (Shaker et al., 2016). Their novelty comes from recombining local patterns rather than reproducing whole levels. The tradeoff is that they capture local structure well but global coherence poorly, and difficulty conditioning becomes indirect, typically a separate model per class or a generate and filter step using the scorer. They also fall outside the GAN, VAE, and Transformer families that frame this project, so they fit better as a supplementary baseline or a component in a larger system than as the primary model. Within those families, a VAE is the more regularized option for small data, since its latent bottleneck and KL term act as a prior that curbs memorization.

Foundation model fine-tuning. A second direction addresses capacity from the other end: rather than training a 3.28 million parameter Transformer from scratch, fine-tune a pretrained foundation model, specifically a MarioGPT style GPT-2 fine-tune, which is the current state of the art for this task (Sudhakaran et al., 2023). Pretrained sequence priors should improve structural coherence and controllability and converge with less data. They would not necessarily solve the core problem, however, because memorization on roughly 400 examples is a function of capacity relative to data, and a much larger pretrained model can overfit a tiny dataset faster. This is effectively a new project rather than an enhancement of the current deliverable, since it requires a different architecture, tokenizer, and training stack and a model two orders of magnitude larger.

Multiple model system rather than one model. The experiments together suggest that a single small model cannot both condition on difficulty and avoid memorizing on this data, which is an argument for a system of components rather than one model carrying the whole task. Decomposing generation into several trained sub models, such as one for layout and one for tile detail, would make matters worse here, since each sub model would train on an even smaller slice of the corpus. The more promising version pairs one generator with components that need no training data: generate a batch, then filter and rank the outputs with cheap rule based tools, the structural validity check, the difficulty scorer, a novelty threshold against the training set, and a playability test. These filters sidestep the data constraint because they are not learned from the small corpus, and they turn generation into a generate, score, validate, and retry loop that enforces quality at the output stage (Shaker et al., 2016). The artifacts this project already produces, a generator, a difficulty scorer, and a structural validity check, are the building blocks for such a system.

References

- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations*. <https://arxiv.org/abs/1904.09751>
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. <https://arxiv.org/abs/1711.05101>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the 2019 ACM*

Conference on Fairness, Accountability, and Transparency (FAT '19)* (pp. 220–229). ACM. <https://doi.org/10.1145/3287560.3287596>

Shaker, N., Togelius, J., & Nelson, M. J. (2016). *Procedural content generation in games*. Springer. <https://doi.org/10.1007/978-3-319-42716-4> (freely available at <https://www.pcgbook.com/>)

Sudhakaran, S., González-Duque, M., Freiberger, M., Glanois, C., Najarro, E., & Risi, S. (2023). MarioGPT: Open-ended text2level generation through large language models. In *Advances in Neural Information Processing Systems* (Vol. 36). <https://arxiv.org/abs/2302.05981>

Summerville, A., Snodgrass, S., Mateas, M., & Ontañón, S. (2016). The VGLC: The video game level corpus. In *Proceedings of the 7th Workshop on Procedural Content Generation*. <https://arxiv.org/abs/1606.07487>

Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A., & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3), 257–270. <https://doi.org/10.1109/TG.2018.2846639> (freely available at <https://arxiv.org/abs/1702.00539>)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30). <https://arxiv.org/abs/1706.03762>

Withington, O., Cook, M., & Tokarchuk, L. (2024). On the evaluation of procedural level generation systems. In *Proceedings of the 19th International Conference on the Foundations of Digital Games*. <https://doi.org/10.1145/3649921.3650016>